Git CLI

Commands, branching, recovery, and safe remote workflows

GET STARTED

`git init`
Initialize a new repository.

`git clone <url>`
Clone a remote repository.

`git status -sb`
Compact status with branch and upstream.

`git remote -v`
Inspect fetch and push remotes.

`git config --global -l`
Inspect global Git configuration.

INSPECT CHANGES

`git diff`
Review unstaged changes.

`git diff --staged`
Review staged changes.

`git diff --check`
Catch whitespace errors.

`git log --oneline --graph --decorate -15`
Compact commit graph.

`git show HEAD`
Inspect the most recent commit.

`git log --follow -- <file>`
Trace changes through file renames.

`git blame -L 10,30 <file>`
Identify authors of a line range.

`git grep -n "pattern"`
Search tracked files.

`git log -S"symbol" --all`
Find commits adding/removing a string.

STAGE AND COMMIT

`git add <file>`
Stage a file.

`git add -p`
Stage selected hunks interactively.

`git restore --staged <file>`
Unstage without changing working file.

`git commit -m "message"`
Commit staged changes.

`git commit --amend`
Replace the latest local commit; rewrites ID.

`git mv old new`
Move a tracked file.

`git rm --cached <file>`
Untrack but retain local file.

BRANCHES AND WORKTREES

`git branch -avv`
List local and remote branches.

`git switch <branch>`
Change current branch.

`git switch -c <new>`
Create and switch to branch.

`git branch -d <branch>`
Delete merged local branch.

`git worktree add ../fix -b fix`
Create second checkout on new branch.

`git worktree list`
List active worktrees.

REMOTE SYNCHRONIZATION

`git fetch --prune origin`
Update remote refs; remove stale remote refs.

`git pull --ff-only`
Pull only if fast-forward is possible.

Git CLI

Commands, branching, recovery, and safe remote workflows

`git push -u origin <branch>`
Publish branch and record upstream.

`git push`
Push current branch to tracked upstream.

`git merge origin/main`
Merge remote-tracking main.

`git rebase origin/main`
Replay local commits on new base.

`git rebase --continue`
Continue after resolving conflicts.

`git rebase --abort`
Cancel rebase and restore starting point.

RECOVER AND UNDO

`git stash push -u -m "wip"`
Stash tracked and untracked files.

`git stash list`
Inspect saved stashes.

`git stash pop`
Reapply most recent stash and drop if successful.

`git reflog`
Find previous branch tips and lost commits.

`git switch -c rescue <sha>`
Protect recovered commit in new branch.

`git revert <sha>`
Create inverse commit; safer for shared branches.

`git restore -- <file>`
DISCARDS unstaged edits in tracked file.

`git clean -nd`
Preview untracked deletion.

`git clean -fd`
DELETES untracked files; preview first.

`git reset --hard <sha>`
DESTRUCTIVE: reset HEAD, index, worktree.

MERGE AND CODE ARCHAEOLOGY

`git diff main...feature`
Compare feature to merge-base of main.

`git cherry-pick <sha>`
Apply an individual commit.

`git merge --abort`
Cancel in-progress merge.

`git bisect start`
Begin regression bisection; mark good/bad.

`git bisect reset`
Stop bisection and restore original state.

KEY CONSIDERATIONS

MENTAL MODEL: HEAD is current commit; index is staging; working tree is checked-out files. Fetch does not merge. Rebase and amend change commit IDs; avoid rewriting shared work.

OFFICIAL DOCUMENTATION

`https://git-scm.com/docs`

`https://git-scm.com/cheat-sheet`